

Поиск хайлайтов в видео: итоговый отчет

Дата подготовки: 18 мая 2026

1. Постановка задачи

Задача проекта - временная локализация хайлайтов в видео. На вход подается короткий видеофрагмент и текстовый запрос, на выходе ожидается один или несколько временных интервалов, где содержимое видео соответствует запросу и является релевантным моментом.

В итоговой реализации задача сведена к clip-level классификации. Видео разбивается на двухсекундные клипы, для каждого клипа предсказывается score релевантности, после чего соседние положительные клипы объединяются во временные интервалы. Такой подход позволяет получить полный воспроизводимый ML-пайплайн: подготовка данных, обучение, подбор threshold, оценка качества и анализ ошибок.

2. Датасет QVHighlights

В работе используется QVHighlights - открытый датасет для query-conditioned video moment retrieval и highlight detection. В каждой записи есть видеофрагмент, текстовый запрос и разметка релевантных моментов. Для train/val доступны:

- `qid` - идентификатор запроса;
- `vid` - идентификатор видеофрагмента;
- `query` - текстовый запрос;
- `duration` - длительность фрагмента;
- `relevant\_windows` - ground-truth интервалы в секундах;
- `relevant\_clip\_ids` - идентификаторы релевантных двухсекундных клипов;
- `saliency\_scores` - оценки важности клипов от аннотаторов.

В test split ground truth не раскрыт, поэтому финальная численная оценка проводится на validation split.

split	rows	unique videos	total hours	median duration
train	7218	7100	299.84	150 s
val	1550	1519	64.42	150 s
test	1542	1529	64.06	150 s

Предварительный анализ данных показал, что большинство фрагментов имеют длительность около 150 секунд, а разметка часто содержит один или несколько

релевантных интервалов внутри видео. Проверка качества аннотаций не выявила критических ошибок таймкодов в использованных JSONL-файлах.

3. Moment-DETR features и CLIP embeddings

Moment-DETR - transformer-подход для поиска временных моментов в видео по текстовому запросу. Полная модель принимает последовательность video features и text features, после чего предсказывает временные границы релевантных моментов. В проекте не обучался полный Moment-DETR end-to-end. Вместо этого использованы предрасчитанные features, подготовленные для Moment-DETR.

В финальном pipeline используются только CLIP-признаки:

```
features/  
clip_features/<vid>.npz  
clip_text_features/qid<id>.npz
```

`clip\_features/<vid>.npz` содержит матрицу признаков видео. Для типичного 150-секундного фрагмента это массив размера `(75, 512)`: 75 двухсекундных клипов, каждому клипу соответствует 512-мерный CLIP embedding. Такой embedding является числовым описанием визуального содержимого клипа.

`clip\_text\_features/qid<id>.npz` содержит CLIP embedding текстового запроса, обычно 512-мерный vector `pooler\_output`. Так как CLIP обучен сопоставлять визуальное и текстовое содержимое в общем embedding space, между video embedding и text embedding можно считать cosine similarity.

Если считать эти признаки с нуля, процесс выглядит так:

1. Видео разбивается на короткие клипы.
2. Из каждого клипа выбираются кадры.
3. Кадры проходят через visual encoder CLIP.
4. Frame embeddings агрегируются в один embedding клипа.
5. Текстовый запрос проходит через text encoder CLIP.
6. Для пары clip/query считается similarity в общем embedding space.

В проекте признаки уже предрасчитаны, поэтому обучение не требует хранения и декодирования исходных `.mp4`. Проверка coverage после удаления неиспользуемых feature-папок:

```
clip_features: 12562 .npz files  
clip_text_features: 10310 .npz files  
clip video coverage: 10148/10148  
text feature coverage: 10310/10310
```

4. Выбор подхода

Финальная система реализована как легкий video-aware baseline поверх CLIP features. Такой выбор обусловлен структурой данных и задачей:

- QVHighlights предоставляет разметку на уровне временных окон и clip ids;
- CLIP features дают визуально-текстовое представление без загрузки raw video;
- clip-level классификация естественно использует `relevant\_clip\_ids`;
- объединение соседних положительных клипов дает интервалы, которые можно оценивать через temporal IoU;
- подход воспроизводим на CPU и сохраняет все промежуточные артефакты.

Рассмотренные альтернативы:

- Полный Moment-DETR. Потенциально более сильное решение, так как моделирует последовательность и напрямую предсказывает границы моментов. В рамках проекта выбран более легкий baseline, чтобы получить воспроизводимый полный цикл обучения и оценки.

- Raw-video pipeline. Требуется скачивание и декодирование исходных роликов, что существенно усложняет воспроизведение и увеличивает объем данных.

- Temporal model поверх CLIP embeddings: BiLSTM, TCN или Transformer encoder. Это логичный следующий шаг, но он требует отдельной серии экспериментов.

- Annotation-only baseline. Такой подход проще, но он не использует визуальное содержимое видео, поэтому не подходит как основное решение.

5. Реализованный ML-пайплайн

Пайплайн реализован в `train\_moment\_features\_model.py`.

Основные шаги:

1. JSONL-записи train/val читаются из `data/qvhighlights`.
2. Каждая запись разворачивается в последовательность двухсекундных клипов.
3. Target равен 1, если `clip\_id` входит в `relevant\_clip\_ids`.
4. Для каждого клипа загружается CLIP video feature.
5. Для каждого запроса загружается CLIP text feature.
6. Считается `clip\_text\_cosine` между embedding клипа и embedding запроса.
7. Добавляются временные признаки: `clip\_id`, `clip\_start`, `clip\_end`, относительная позиция клипа внутри видео.
8. Добавляются TF-IDF признаки текста запроса.
9. Обучается линейный классификатор.

10. На validation split подбирается threshold.
11. Последовательные положительные клипы объединяются в predicted windows.
12. Сохраняются модель, метрики, threshold sweep, примеры предсказаний и анализ ошибок.

Финальная команда запуска:

```
python train_moment_features_model.py \  
--max-train-videos 1000 \  
--max-val-videos 300 \  
--classifier sgd \  
--max-iter 30 \  
--video-feature-dirs clip_features
```

Финальный эксперимент выполнен на subset `1000 train / 300 val`. Полный набор features доступен локально, но для итогового запуска выбран ограниченный subset, чтобы сохранить приемлемое время обучения и объем оперативной памяти.

6. Почему используется SGDClassifier

В качестве классификатора выбран `SGDClassifier(loss="log\_loss")`. При такой настройке модель является линейной логистической классификацией, обучаемой стохастическим градиентным спуском.

Выбор `SGDClassifier` обоснован следующими причинами:

- признаков много: CLIP video embedding, CLIP text embedding, TF-IDF запроса, cosine similarity и временные признаки;
- после разворачивания видео в клипы число обучающих объектов быстро растет;
- линейная модель хорошо подходит как baseline для dense embeddings;
- `SGDClassifier` быстрее и легче масштабируется на CPU, чем классическая `LogisticRegression` на больших матрицах;
- `loss="log\_loss"` дает вероятностный score, поэтому можно подбирать threshold для перехода от clip-level scores к временным интервалам;
- `class\_weight="balanced"` компенсирует дисбаланс между релевантными и нерелевантными клипами;
- `early\_stopping=True` снижает риск переобучения и ускоряет обучение.

Таким образом, `SGDClassifier` выбран не как финальная максимально сильная архитектура, а как быстрый, интерпретируемый и воспроизводимый baseline для проверки качества CLIP features в задаче highlight detection.

7. Метрики и протокол оценки

Оценка проводится на validation split, так как в test split отсутствует открытая разметка релевантных окон.

Использованные метрики:

- `clip\_precision`, `clip\_recall`, `clip\_f1` - качество бинарной классификации двухсекундных клипов;
- `clip\_average\_precision` - threshold-free качество ранжирования клипов;
- `segment\_f1\_iou\_0\_3` - F1 по временным интервалам при temporal IoU ≥ 0.3 ;
- `segment\_f1\_iou\_0\_5` - более строгий F1 по интервалам при IoU ≥ 0.5 ;
- `mean\_best\_iou` - средний лучший IoU между ground truth окнами и предсказанными окнами.

Протокол оценки:

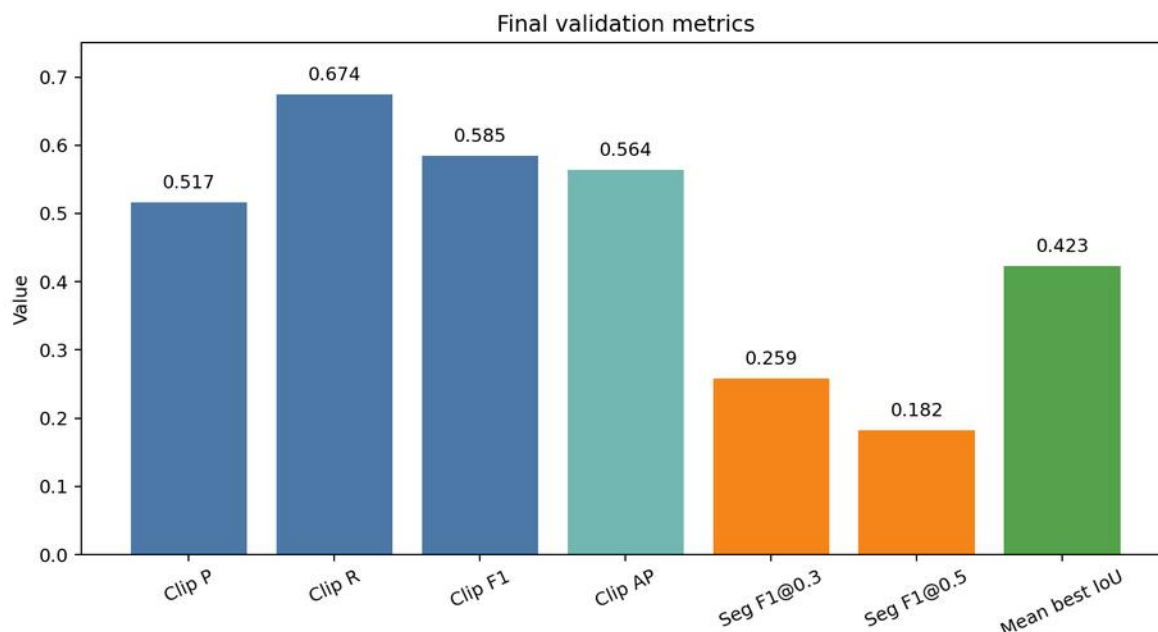
1. Обучение проводится на первых 1000 train query/video записей.
2. Валидация проводится на первых 300 val query/video записей.
3. Для threshold от 0.05 до 0.95 считаются clip-level и segment-level метрики.
4. Финальный threshold выбирается по максимальному `segment\_f1\_iou\_0\_5`.
5. Для анализа ошибок сохраняются худшие случаи по `mean\_best\_iou`.

8. Результаты

Финальные метрики на subset `1000 train / 300 val`:

Метрика	Значение
threshold	0.150
clip precision	0.517
clip recall	0.674
clip F1	0.585
clip AP	0.564
segment F1 @ IoU 0.3	0.259
segment F1 @ IoU 0.5	0.182
mean best IoU	0.423

Итоговые метрики

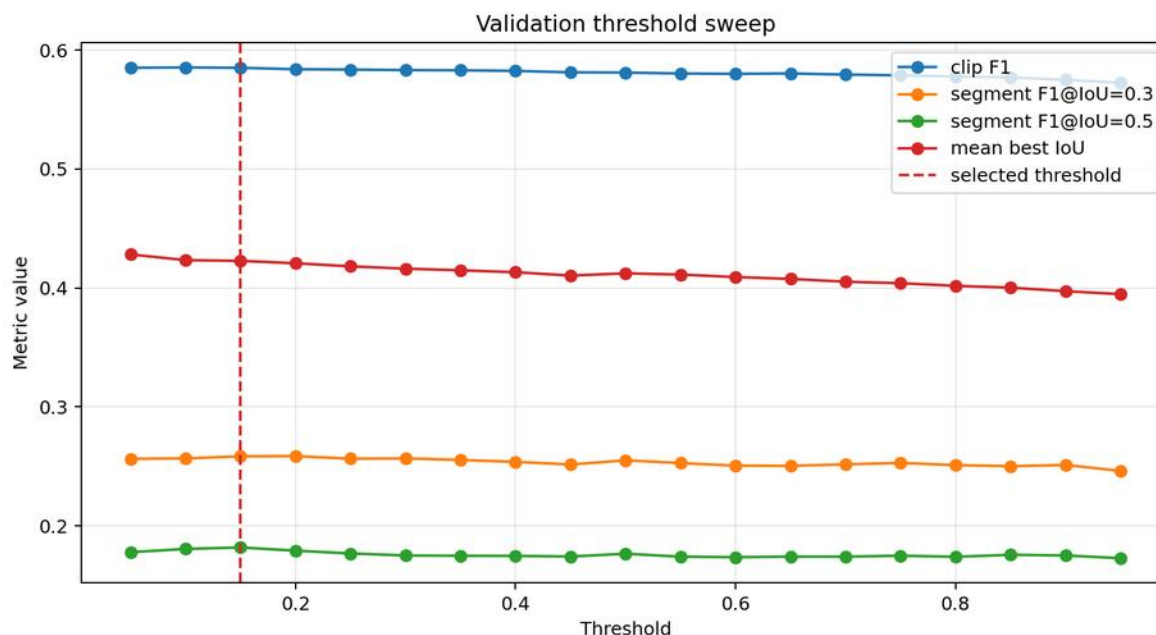


На clip-level уровне модель показывает устойчивый сигнал: ``clip_AP=0.564``. Это означает, что CLIP embeddings и query-video similarity помогают ранжировать двухсекундные клипы по релевантности запросу. Segment-level метрики ниже, потому что точное восстановление границ временных интервалов сложнее, чем классификация отдельных клипов.

9. Подбор threshold

Финальный threshold равен ``0.15``. Он выбран по ``segment_f1_iou_0_5``, так как целевая задача состоит в поиске временных интервалов, а не только отдельных положительных клипов.

Threshold sweep



На threshold sweep видно, что снижение threshold увеличивает recall, но создает больше ложных положительных интервалов. Повышение threshold улучшает precision, но чаще приводит к пропуску релевантных окон. Threshold `0.15` дает лучший компромисс по строгой segment-level метрике.

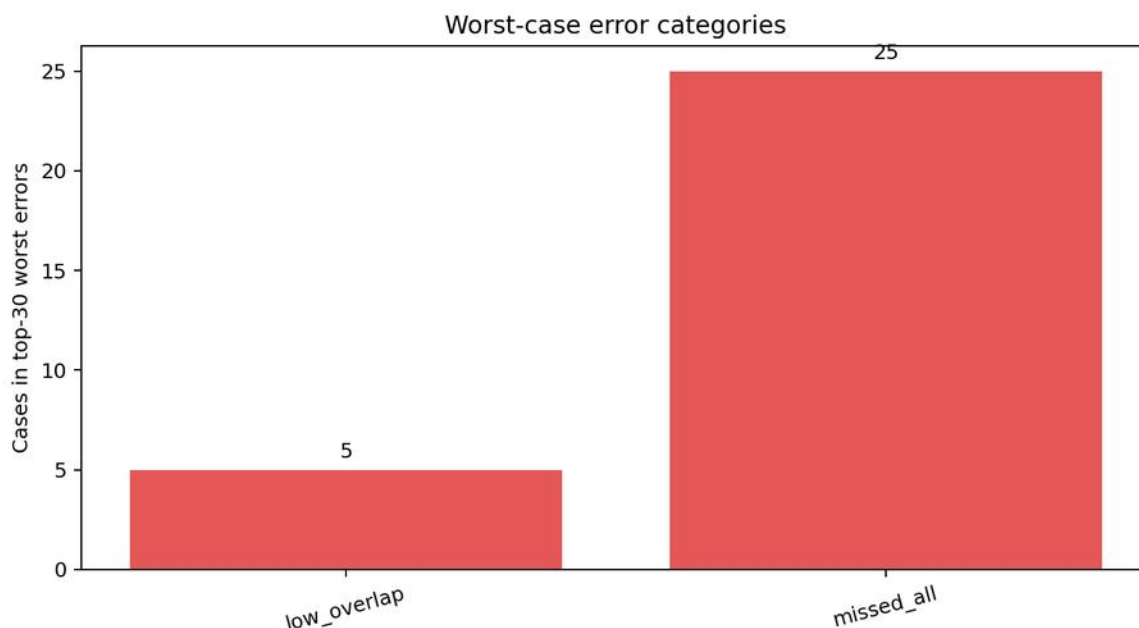
10. Анализ ошибок

Для анализа ошибок сохраняется файл

`outputs/stage2\_moment\_features/error\_analysis.json`. Для каждого случая фиксируются query, video id, ground-truth окна, predicted windows, top clips и категория ошибки.

Категории ошибок:

- `missed\_all` - не найдено ни одного окна;
- `low\_overlap` - предсказанные окна есть, но плохо пересекаются с ground truth;
- `over\_prediction` - предсказание слишком широкое;
- `under\_prediction` - предсказание слишком короткое;
- `acceptable\_overlap` - приемлемое пересечение.



Основной источник ошибок - независимое принятие решений по клипам. Модель может правильно найти часть релевантной области, но разбить длинный момент на несколько фрагментов или сместить границы относительно ground truth. Это особенно заметно на длинных окнах, где требуется учитывать temporal context между соседними клипами.

11. Демонстрация работы модели

Для проверки работы сохраненной модели подготовлен notebook `demo_highlight_model.ipynb`. В нем выполняются следующие шаги:

1. Загружается `moment_features_highlight_model.joblib`.
2. Загружаются validation-записи и CLIP features.
3. Для клипов считаются вероятности через `predict_proba`.
4. Scores переводятся во временные интервалы через threshold `0.15`.
5. Predicted windows сравниваются с ground truth windows на timeline-графиках.

На demo subset из 40 validation-записей было построено 2988 clip-level примеров.

Полученные demo-метрики:

Метрика	Значение
clip F1	0.666
clip AP	0.642
segment F1 @ IoU 0.5	0.163
mean best IoU	0.456

Demo subset меньше финального validation subset, поэтому эти значения не

заменяют основные результаты из раздела 8. Их назначение - показать, что сохраненная модель действительно загружается, считает scores и строит временные интервалы для новых validation-примеров.

12. Ограничения

Ограничения итогового решения:

- обучение выполнено на subset, а не на полном train split;
- модель принимает решения на уровне отдельных клипов;
- temporal dependencies между соседними клипами явно не моделируются;
- saliency scores используются для анализа данных, но не как soft labels;
- постобработка интервалов ограничена простым объединением соседних положительных клипов.

Эти ограничения объясняют разрыв между clip-level и segment-level метриками: модель уже умеет находить релевантные клипы, но пока не всегда точно восстанавливает границы временных моментов.

13. Пути доработки

Возможные улучшения:

- обучить temporal model поверх последовательности CLIP embeddings;
- использовать BiLSTM, TCN или Transformer encoder для учета соседних клипов;
- перейти от hard labels к soft labels на основе `saliency\_scores`;
- улучшить постобработку: merge gaps, min/max duration, temporal NMS;
- подобрать threshold не глобально, а с учетом статистик конкретного видео или запроса;
- провести полный train/val запуск на большем объеме данных.

14. Выводы

В результате реализован воспроизводимый baseline для поиска хайлайтов в видео:

- использован QVHighlights;
- применены предрасчитанные CLIP video/text features из Moment-DETR pipeline;
- реализовано обучение clip-level классификатора;
- реализованы clip-level и segment-level метрики;
- сохранены модель, метрики, threshold sweep, примеры предсказаний и анализ ошибок;

- подготовлен notebook для демонстрации работы сохраненной модели.

Полученный результат показывает, что CLIP embeddings дают полезный сигнал для поиска релевантных клипов по текстовому запросу. Основное направление улучшения качества связано с моделированием temporal context и более точной постобработкой границ интервалов.

15. Воспроизводимость

Минимальная структура проекта:

```
data/qvhighlights/  
highlight_train_release.jsonl  
highlight_val_release.jsonl  
highlight_test_release.jsonl  
features/  
clip_features/  
clip_text_features/  
outputs/stage2_moment_features/  
metrics.json  
threshold_sweep.csv  
val_predictions_examples.json  
error_analysis.json  
moment_features_highlight_model.joblib
```

Команды воспроизведения:

```
python -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt  
python scripts/inspect_moment_features.py  
python train_moment_features_model.py \  
--max-train-videos 1000 \  
--max-val-videos 300 \  
--classifier sgd \  
--max-iter 30 \  
--video-feature-dirs clip_features
```